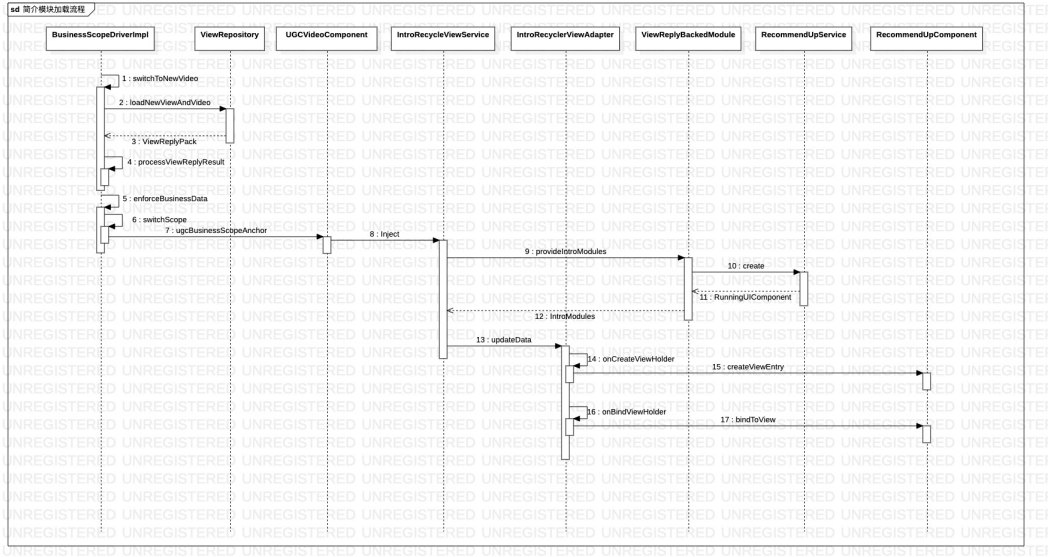


详情页简介流程

1.流程图



2.Component展示状态并传递事件

- ```
1 /**
2 * 基本思想：数据向下，事件向上
3 *
4 * 数据向下：component接收来自service中创建的视图状态数据并进行展示，这些数据可以
5 * 是不变的，也可以是可变的（根据不同的产品需求进行设计）；
6 * 事件向上：component向service发送自身的视图事件，例如点击、曝光和滑动事件，由se
7 * rvice处理事件；
8 *
9 * component状态数据：
10 * 状态数据是描述component中可闭环的模块状态，通常情况下一个component中只有一个状
11 * 态数据，如模块标题，子标题，封面等；
12 * 但是也可能存在一个component有两个或者多个独立闭环模块状态数据，所以并不会限制co
13 * mponent中只存在一个状态数据
14 * 注：一定是视图状态数据，而非网络或其他接口数据
15 *
16 * component交互事件：
17 * 通常情况下component中向上分发的事件是用户交互过程中产生的滑动，点击以及控件曝光
18 * 等事件；
19 * 一定是视图本身产生的事件，而非逻辑代码，如产品需要在某个控件曝光时上报一个埋点，
20 * 那么对外分发的是控件曝光的事件（onXXXShow()），
21 * 然后在service中针对曝光事件进行埋点上报处理(reportXXXShow())，而不是在视图控件
22 * 曝光时直接调用曝光处理的方法；
23 * 每个事件都应该独立，不应该和其他事件有交集，应该独立进行分发；
24 *
25 */
```

```
// demo1: component 仅展示不可变的文案
class TheseusSimpleComponent(
 private val text: String,
) : UIComponent<UIComponent.ViewEntry.ViewViewEntry<TextView>> {

 override fun createViewEntry(
 context: Context,
 parent: ViewGroup?,
): UIComponent.ViewEntry.ViewViewEntry<TextView> {
 val view = TextView(parent?.context)
 view.layoutParams = ViewGroup.LayoutParams(
 ViewGroup.LayoutParams.MATCH_PARENT,
 ViewGroup.LayoutParams.WRAP_CONTENT,
)
 return UIComponent.ViewEntry.ViewViewEntry(view)
 }

 override suspend fun bindToView(viewEntry:
 UIComponent.ViewEntry.ViewViewEntry<TextView>) {
 viewEntry.value.text = text
 }
}
```

```

// demo2: component 拥有可变的狀態
class TheseusStateComponent(
 private val state: StateFlow<SimpleState>,
) : UIComponent<UIComponent.ViewEntry.ViewViewEntry<TextView>> {

 override fun createViewEntry(
 context: Context,
 parent: ViewGroup?,
): UIComponent.ViewEntry.ViewViewEntry<TextView> {
 val view = TextView(parent?.context)
 view.layoutParams = ViewGroup.LayoutParams(
 ViewGroup.LayoutParams.MATCH_PARENT,
 ViewGroup.LayoutParams.WRAP_CONTENT,
)
 return UIComponent.ViewEntry.ViewViewEntry(view)
 }

 override suspend fun bindToView(viewEntry:
 UIComponent.ViewEntry.ViewViewEntry<TextView>) {
 val view = viewEntry.value
 coroutineScope {
 launch {
 state.collectLatest {
 view.text = it.text
 view.isSelected = it.isSelected
 }
 }
 }
 }
}

data class SimpleState(
 val text: String,
 val isSelected: Boolean,
)
}

```

```
// demo3: component 可以拥有可变的狀態，並且會向外分發component本身的各种事件
class TheseusStateWithEventComponent(
 private val state: StateFlow<SimpleState>,
 private val onTextClick: () -> Unit,
 private val onTextShow: () -> Unit,
) : UIComponent<UIComponent.ViewEntry.ViewViewEntry<TextView>> {

 override fun createViewEntry(
 context: Context,
 parent: ViewGroup?,
): UIComponent.ViewEntry.ViewViewEntry<TextView> {
 val view = TextView(parent?.context)
 view.layoutParams = ViewGroup.LayoutParams(
 ViewGroup.LayoutParams.MATCH_PARENT,
 ViewGroup.LayoutParams.WRAP_CONTENT,
)
 return UIComponent.ViewEntry.ViewViewEntry(view)
 }

 override suspend fun bindToView(viewEntry:
 UIComponent.ViewEntry.ViewViewEntry<TextView>) {
 val view = viewEntry.value
 view.collectExposureLayoutInfo(
 ExposeFactory.newExposureEntry {
 onTextShow()
 },
)

 coroutineScope {
 launch {
 state.collectLatest {
 view.text = it.text
 view.isSelected = it.isSelected
 }
 }
 launch {
 view.onClickEvent().collectLatest {
 onTextClick()
 }
 }
 }
 }
}

data class SimpleState(
 val text: String,
 val isSelected: Boolean,
)
}
```

```

// demo4: component 拥有多个可变的狀態，并且会向外分发component本身的各种事件
class TheseusMultiStateWithEventComponent(
 private val simpleState: StateFlow<SimpleState>,
 private val multiState: StateFlow<MultiState>,
 private val onTextClick: () -> Unit,
 private val onTextShow: () -> Unit,
) : UIComponent<UIComponent.ViewEntry.ViewViewEntry<TextView>> {

 override fun createViewEntry(
 context: Context,
 parent: ViewGroup?,
): UIComponent.ViewEntry.ViewViewEntry<TextView> {
 val view = TextView(parent?.context)
 view.layoutParams = ViewGroup.LayoutParams(
 ViewGroup.LayoutParams.MATCH_PARENT,
 ViewGroup.LayoutParams.WRAP_CONTENT,
)
 return UIComponent.ViewEntry.ViewViewEntry(view)
 }

 override suspend fun bindToView(viewEntry:
 UIComponent.ViewEntry.ViewViewEntry<TextView>) {
 val view = viewEntry.value
 view.collectExposureLayoutInfo(
 ExposeFactory.newExposureEntry {
 onTextShow()
 },
)

 coroutineScope {
 launch {
 simpleState.collectLatest {
 view.text = it.text
 view.isSelected = it.isSelected
 }
 }
 launch {
 multiState.collectLatest {
// view2.text = it.text
// view2.isSelected = it.isSelected
 }
 }

 launch {
 view.onClickEvent().collectLatest {
 onTextClick()
 }
 }
 }
 }
}

```

```

 }
}

data class SimpleState(
 val text: String,
 val isSelected: Boolean,
)

data class MultiState(
 val text: String,
 val isSelected: Boolean,
)
}

```

```

// demo5: component 的入参过多时, 可以考虑使用view model
class TheseusViewModelComponent(
 private val viewModel: ViewModel,
) : UIComponent<UIComponent.ViewEntry.ViewViewEntry<TextView>> {

 override fun createViewEntry(
 context: Context,
 parent: ViewGroup?,
): UIComponent.ViewEntry.ViewViewEntry<TextView> {
 val view = TextView(parent?.context)
 view.layoutParams = ViewGroup.LayoutParams(
 ViewGroup.LayoutParams.MATCH_PARENT,
 ViewGroup.LayoutParams.WRAP_CONTENT,
)
 return UIComponent.ViewEntry.ViewViewEntry(view)
 }

 override suspend fun bindToView(viewEntry:
 UIComponent.ViewEntry.ViewViewEntry<TextView>) {
 val view = viewEntry.value
 view.collectExposureLayoutInfo(
 ExposeFactory.newExposureEntry {
 viewModel.onTextShow()
 },
)

 coroutineScope {
 launch {
 viewModel.simpleState.collectLatest {
 view.text = it.text
 view.isSelected = it.isSelected
 }
 }
 }
 launch {

```

```

 viewModel.multiState.collectLatest {
// view2.text = it.text
// view2.isSelected = it.isSelected
 }
 }

 launch {
 view.onClickEvent().collectLatest {
 viewModel.onTextClick()
 }
 }
}

data class SimpleState(
 val text: String,
 val isSelected: Boolean,
)

data class MultiState(
 val text: String,
 val isSelected: Boolean,
)

interface ViewModel {
 val simpleState: StateFlow<SimpleState>

 val multiState: StateFlow<MultiState>

 val onTextClick: () -> Unit

 val onTextShow: () -> Unit
}

```

### 3.Service创建Component，为Component提供展示状态并处理Component的事件

```

@DetailBizScope
class TheseusDemoService @Inject constructor(
 @DetailBizCoroutineScope private val scope: CoroutineScope,
 private val demoRepository: TheseusDemoRepository,
 private val recycleViewService: IntroRecycleViewService,
) {

```

/\*\*

\* 1.普通的component，view接口返回时就会直接显示在列表中

```

*
* 通过依赖注入调用service中的create方法，创建component
*
* [com.bilibili.ship.theseus.united.page.intro.di.IntroModuleListMapper],
* [com.bilibili.ship.theseus.ugc.intro.UGCIntroModuleParserProvider]
*/
fun create(): RunningUIComponent {
 // 可变视图状态1
 val simpleStateFlow = MutableStateFlow(
 TheseusViewModelComponent.SimpleState(
 "text1",
 false,
),
)

 // 可变视图状态2
 val multiStateFlow = MutableStateFlow(
 TheseusViewModelComponent.MultiState(
 "text2",
 false,
),
)

 // 处理交互事件
 fun handleClickEvent() {
 // report show and do something
 infoLog("handleClickEvent")
 demoRepository.loadData()
 }

 // 处理曝光事件
 fun handleShowEvent() {
 // report click and do something
 infoLog("handleShowEvent")
 }

 // 创建视图模型
 val viewModel = object : TheseusViewModelComponent.ViewModel {
 override val simpleState = simpleStateFlow

 override val multiState = multiStateFlow

 override val onTextClick = { handleClickEvent() }

 override val onTextShow = { handleShowEvent() }
 }

 return RunningUIComponent(TheseusViewModelComponent(viewModel)) {
 /**
 * [RunningUIComponent]

```



\* 拥有一个伴随其显示过程的作用域，该作用域在component插入简介列表时启动，在移除简介列表时取消；

```
*/
coroutineScope {
 /**
 * 接收repository模块数据并更新到component
 */
 launch {
 demoRepository.demoDataFlow.collectLatest {
 simpleStateFlow.value = simpleStateFlow.value.copy(
 text = "update text",
)
 }
 }

 /**
 * 接收repository消息数据并处理（更新到component或者进行其他处理）
 */
 launch {
 demoRepository.showPromptFlow.collectLatest {
 infoLog("show prompt")
 // 1. update to component
 multiStateFlow.value = multiStateFlow.value.copy(
 text = "prompt text",
)

 // 2. do something
 }
 }
}

/**
 * 2.后显模块，在view接口返回时并不会直接显示，而是先使用一个占位模块（模块顺序
由view接口控制，等到特定时机进行替换）
 *
 * [PlaceholderComponent]
 * [IntroRecyclerViewService.replaceComponent]
 *
 * 如果模块替换展示时需要动画，可以使用
[com.bilibili.ship.theseus.united.page.intro.module.fold.FoldAnimationComponent]
 */
fun createReplace(): RunningUIComponent {
 val placeholder = PlaceholderComponent()

 return RunningUIComponent(placeholder) {
 coroutineScope {
 launch {
 delay(10 * 1000)
```

```
 delay(1000 - 1000),
 recycleViewService.replaceComponent(
 RunningUIComponent(TheseusSimpleComponent("text")),
 placeholder,
)
 }
}
}
}
```

#### 4.Repository处理接口请求和消息数据，并提供相应的Flow

```

/**
 * 处理接口请求和消息数据，并提供相应的Flow
 */
@DetailBizScope
class TheseusDemoRepository @Inject constructor(
 @DetailBizCoroutineScope private val scope: CoroutineScope,
) {

 private val _showPromptFlow = MutableSharedFlow<Unit>(extraBufferCapacity = 1,
onBufferOverflow = BufferOverflow.DROP_LATEST)
 val showPromptFlow: SharedFlow<Unit> = _showPromptFlow.asSharedFlow()

 /**
 * repository作为service之间的沟通桥梁，其他service也可以通过showPrompt方法触发
demo service的某个行为
 */
 fun showPrompt() {
 _showPromptFlow.tryEmit(Unit)
 }

 private val _demoDataFlow = MutableStateFlow(DemoData(0L))
 val demoDataFlow: StateFlow<DemoData> = _demoDataFlow.asStateFlow()

 /**
 * 加载模块数据
 */
 fun loadData() {
 scope.launch {
 infoLog("start load data")
 withContext(Dispatchers.IO) {
 delay(10 * 1000)
 _demoDataFlow.value = DemoData(1)
 }
 }
 }

 data class DemoData(
 val id: Long,
)
}

```